Práctica 4: Gestión de imágenes y contenedores

Gestión de Imágenes y Contenedores en Docker

📅 Semana: 4

👩‍🏫 Docente: Leidy Mariani Dorado

⏱ Duración: 2 horas

👥 Modalidad: Trabajo en equipos

🎯 Objetivo de la sesión

Al finalizar la práctica, el estudiante será capaz de:

- Diferenciar una imagen de un contenedor.
- Descargar imágenes desde Docker Hub.
- Crear, detener, reiniciar y eliminar contenedores.
- Analizar el consumo y el estado de los contenedores.
- Comprender la relación entre contenedores y sistemas distribuidos.

🔹 PARTE 1 – Exploración de Imágenes

✅ Actividad 1: Ver imágenes locales

Ejecutar:

docker images

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\MATC\Desktop\cIase6> docker images

IMAGE                   ID            DISK USAGE   CONTENT SIZE   EXTRA
api-node-demo:latest    1f49cf4b1374       205MB         49.9MB    U
mi_sitio_web:latest     7005cc284e7b       237MB         62.9MB    U
mysql:latest            e5dc14f6e01c      1.27GB          283MB    U
nginx:latest            0236ee02dcbc       240MB         65.8MB    U
sitio_web2:latest       fbafe37cd83d       237MB         62.9MB
sitioweb:latest         5d28c01db93a       237MB         62.9MB
PS C:\Users\MATC\Desktop\cIase6>
```

Respondan en equipo:

- **¿Qué es el IMAGE ID?**

  Es el identificador único que determina la imagen (receta o plantilla)

- **¿Qué significa TAG?**

  La etiqueta, sobrenombre o alias que se le añade al nombre de una imagen para identificar una versión, entre otras.

- **¿Qué representa SIZE?**

  Es el tamaño o espacio total en disco que ocupa una imagen.

- **¿Una imagen se está ejecutando?**

  No, una imagen no se ejecuta, lo que se ejecuta es el contenedor.

✅ Actividad 2: Descargar nuevas imágenes

Cada equipo debe descargar una imagen diferente:

docker pull redis

O también pueden usar:

docker pull mongo
docker pull httpd
docker pull postgres

REDIS

```
PS C:\Users\MATC\Desktop\cIase6> docker pull redis
Using default tag: latest
latest: Pulling from library/redis
214ecf6e5c87: Pull complete
89b1b12d7109: Pull complete
141e1fc22067: Pull complete
f25f5c85b90d: Pull complete
ae77945b0371: Pull complete
4f4fb700ef54: Pull complete
e798a26a6510: Download complete
4a4db1a35d5b: Download complete
Digest: sha256:1c054d54ecd1597bba52f4304bca5afbc5565ebe614c5b3d7dc5b7f8a0cd768d
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
PS C:\Users\MATC\Desktop\cIase6>
```

Luego, ejecuten nuevamente:

docker images

```
PS C:\Users\MATC\Desktop\cIase6> docker images

IMAGE                   ID            DISK USAGE   CONTENT SIZE   EXTRA
api-node-demo:latest    1f49cf4b1374       205MB         49.9MB    U
mi_sitio_web:latest     7005cc284e7b       237MB         62.9MB    U
mysql:latest            e5dc14f6e01c      1.27GB          283MB    U
nginx:latest            0236ee02dcbc       240MB         65.8MB    U
redis:latest            1c054d54ecd1       204MB         55.3MB
sitio_web2:latest       fbafe37cd83d       237MB         62.9MB
sitioweb:latest         5d28c01db93a       237MB         62.9MB
PS C:\Users\MATC\Desktop\cIase6>
```

Reflexionar:

- ¿Qué cambió?

  Se descargó la imagen de **redis** y quedó guardada localmente con un id, ocupando 204 mb de disco y 53 mb de contenido.

- ¿El servicio ya está funcionando?

  Aún no, es solo una imagen. Para su funcionamiento se debe crear un contenedor a partir de esta misma.

🔹 PARTE 2 – Gestión de Contenedores

✅ Actividad 3: Crear un contenedor

Ejemplo con nginx:

docker run -d --name servidor-web -p 8081:80 nginx

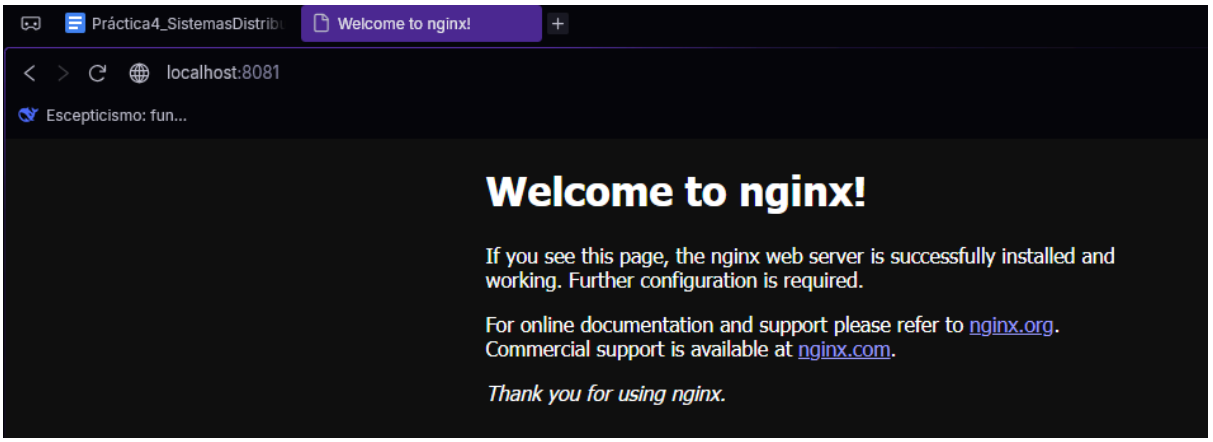
```
PS C:\Users\MATC\Desktop\cIase6> docker run -d --name servidor-web -p 8081:80 nginx
9512b6ee39e03d8bb01916f9954e9fd932dc9ec7ca9e9ee2808c7a7b08182b36
```

Verificar:

docker ps

```
PS C:\Users\MATC\Desktop\cIase6> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                      NAMES
9512b6ee39e0   nginx     "/docker-entrypoint.…"   About a minute ago   Up About a minute   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp   servidor-web
PS C:\Users\MATC\Desktop\cIase6>
```

Abrir el navegador: <http://localhost:8081>



✅ Actividad 4: Administrar el contenedor

Detener:

docker stop servidor-web

```
PS C:\Users\MATC\Desktop\cIase6> docker stop servidor-web
servidor-web
PS C:\Users\MATC\Desktop\cIase6>
```

Ver todos los contenedores:

docker ps -a

```
PS C:\Users\MATC\Desktop\cIase6> docker ps -a
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS                        PORTS                    NAMES
9512b6ee39e0   nginx                 "/docker-entrypoint.…"   7 minutes ago   Exited (0) About a minute ago                          servidor-web
f5ad4a128735   api-node-demo         "docker-entrypoint.s…"   41 hours ago    Exited (255) 48 minutes ago   0.0.0.0:3000->3000/tcp   gifted_lewin
9fbd0462000c   mi_sitio_web:latest   "/docker-entrypoint.…"   2 days ago      Exited (0) 43 hours ago                                tender_antonelli
5d872c081a6d   nginx                 "/docker-entrypoint.…"   2 days ago      Exited (0) 2 days ago                                  compose-1-web-1
d3c2b5e6d1b1   mysql                 "docker-entrypoint.s…"   2 days ago      Exited (137) 2 days ago                                compose-1-db-1
PS C:\Users\MATC\Desktop\cIase6>
```

Reiniciar:

docker start servidor-web

```
PS C:\Users\MATC\Desktop\cIase6> docker start servidor-web
servidor-web
PS C:\Users\MATC\Desktop\cIase6>
```

Eliminar:

docker rm -f servidor-web

```
PS C:\Users\MATC\Desktop\cIase6> docker rm -f servidor-web
servidor-web
PS C:\Users\MATC\Desktop\cIase6>
```

🔹 PARTE 3 – Análisis Conceptual

Completar esta tabla:

Concepto	Imagen	Contenedor
¿Es ejecutable?	No, es estático	Si, es la imagen corriendo
¿Ocupa recursos en ejecución?	No, solo espacio en el disco	Si, CPU, RAM y RED
¿Puede modificarse?	No	Si
¿Puede eliminarse?	Si	Si

🔹 PARTE 4 – Reto en Equipo

Cada grupo debe:

1. Ejecutar un contenedor diferente.

   MONGO

```
PS C:\Users\MATC\Desktop\cIase6> docker pull mongo
Using default tag: latest
latest: Pulling from library/mongo
8d23bbf2b216: Pull complete
427c118b2079: Pull complete
e79e499203c9: Pull complete
01d7766a2e4a: Pull complete
e46eb2190ffc: Pull complete
cea2e6a72e6b: Pull complete
f97b55066391: Pull complete
fbfd783f7a59: Pull complete
4eeb70347fbd: Download complete
bae1425230be: Download complete
Digest: sha256:474f5c3bf0e355bb97dafda730e725169a4d51c5578abf7be9ec7ad3fdee4481
Status: Downloaded newer image for mongo:latest
docker.io/library/mongo:latest
PS C:\Users\MATC\Desktop\cIase6>
```

```
PS C:\Users\MATC\Desktop\cIase6> docker run -d --name practica4 -p 8082:80 mongo
a57d0a753662d575a1a306353d46193a4bc1e2243c576b712ca6aa6154906925
PS C:\Users\MATC\Desktop\cIase6>
```

☐	●	practica4	a57d0a753662	mongo	8082:80	0.46%	120.4M

2. **Mostrar que está activo.**

```
PS C:\Users\MATC\Desktop\cIase6> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                      NAMES
a57d0a753662   mongo     "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes   0.0.0.0:8082->80/tcp, [::]:8082->80/tcp    practica4
PS C:\Users\MATC\Desktop\cIase6>
```

3. **Explicar:**
   - **¿Qué tipo de servicio es?**

     Sistema de Base de Datos no relaciones, orientado a documentos.

   - **¿En qué tipo de sistema distribuido podría usarse?**

     Microservicios: cada servicio necesita su propia BD.

     Réplicas (24/7): Al fallar en un servidor, hay otro u otros donde está respaldada la información (sistemas bancarios o de salud).

     Big Data: Datos masivos.

   - **¿Qué pasaría si ese servicio falla?**

     Interrupción del servicio: errores de “no se pudo conectar a la BD”

     Inaccesibilidad a la información: si no se crean réplicas.

     No lectura ni almacenamiento de información en la aplicación.

🔹 PARTE 5 – Análisis Crítico

Responder:

- **¿Cuál es la diferencia real entre imagen y contenedor?**

  La imagen es como una receta y el contenedor es el plato de comida creado a partir de esa receta, la imagen no cambia ni consume recursos al ser estática (sólo lectura). El contenedor es la imagen corriendo, tiene su propio procesamiento (lectura y escritura). Se pueden crear diversos contenedores diferentes solo requiriendo una imagen y van a funcionar de manera independiente.

- **¿Por qué Docker facilita la escalabilidad?**

  Docker facilita la escalabilidad porque:

    - Permite replicar servicios rápidamente.
    - Los contenedores arrancan en segundos.
    - Usa menos recursos que máquinas virtuales.
    - Es portable entre entornos.

- **¿Cómo se relaciona esto con microservicios?**

  Cada microservicio vive en su propio contenedor, piezas pequeñas, independientes y especializadas. La base de datos en un contenedor, las APIs en otro, el servidor web también aparte, precisamente aquí encontramos la relación con los microservicios, cada contenedor se puede actualizar, modificar, reiniciar o escalar independientemente sin afectar otro módulo del sistema ni detener funcionalidades, que es lo que justamente busca un arquitectura basada en microservicios.

- **¿Qué ventajas ofrece frente a instalar software directamente?**

    - Evita conflictos entre programas
    - Funciona en cualquier computador
    - Instalación más rápida
    - Facilidad en compartir aplicaciones
    - Escalabilidad en aplicaciones
    - Sistema limpio, sin rastros que afecten al sistema.

CONCLUSIONES

1. Docker no solo optimiza el uso de hardware frente a las máquinas virtuales tradicionales, sino que garantiza la portabilidad del software, asegurando que una aplicación se comporte igual en cualquier computador.
2. Docker permite que cada componente de un sistema (base de datos, web, API) se encuentre en su propio entorno aislado, garantizando que si un servicio falla o necesita una actualización, los demás sigan funcionando sin interrupciones.
3. El contenedor se comporta igual en Windows, Linux o cualquier servidor en la nube, o sistema operativo,asegurando un despliegue sin errores.
4. Docker permite una recuperación ante fallos en segundos.
5. Docker permite el versionamiento en cuestión de segundos.

📌 Entregable de la clase

Cada equipo debe subir al repositorio:

- Capturas de comandos ejecutados.
- Respuestas a las preguntas.
- Tabla comparativa completada.
- Conclusión grupal (mínimo 5 líneas).